

WEEK 2 SESSION 3

Enterprise Data Engineering, Analytics & AI

Entity-Relationship (ER) Diagrams

Designing data structures that power real-world applications

SESSION AGENDA

01

What is an ER Diagram?

Foundation & Purpose

02

Core Building Blocks

Entities, Attributes, Relationships

03

Keys & Identifiers

Primary, Foreign & Composite Keys

04

Relationship Types

One-to-One, Many-to-One, One-to-Many

05

Cardinality & Notation

Crow's Foot, Chen, Min-Max

06

Designing for Applications

Real-World Modeling Best Practices

07

Common Mistakes

Pitfalls & How to Avoid Them

08

Draw.io Lab

Hands-on ER Design Exercise

WHAT IS AN ER DIAGRAM?

Definition: An Entity-Relationship (ER) diagram is a visual blueprint of a database that shows how data entities relate to each other — used by data engineers, architects, and developers to design efficient, scalable databases.



Visual Blueprint

Shows the structure before a single line of SQL is written



Relationships First

Forces you to think about how data connects



Universal Language

Used across industries — banking, healthcare, e-commerce, logistics



Design Tool

Translates directly into table schemas and foreign key constraints

💡 Analogy: Think of an ER diagram like an architectural floor plan — you wouldn't build a house without one!

CORE BUILDING BLOCKS

ENTITY

A real-world object or concept

Represented as a **rectangle** in ER diagrams

Examples:

- Customer
- Product
- Order
- Employee
- Department

Rule: Entities become tables in your database

ATTRIBUTE

Properties that describe an entity

Represented as an **oval/ellipse**

Types:

- Simple: CustomerName
- Composite: FullAddress
- Derived: Age (from DOB)
- Multi-valued: PhoneNumbers

Rule: Attributes become columns in a table

RELATIONSHIP

How entities are connected

Represented as a **diamond/rhombus**

Examples:

- Customer PLACES Order
- Employee MANAGES Dept
- Product BELONGS_TO Category
- Student ENROLLS_IN Course

Rule: Relationships become foreign keys or junction tables

KEYS & IDENTIFIERS

Keys uniquely identify records and establish relationships between tables.

Primary Key (PK)

Uniquely identifies every row in a table. Cannot be NULL or duplicate.

Rules: Must be unique across all rows • Must never be NULL • Should be stable — avoid using changeable values • Ideally a single column (simple key)

Example: *CustomerID, OrderID, ProductID, EmployeeID*

Foreign Key (FK)

A column that references the Primary Key of another table. Enforces referential integrity.

Rules: Must match a valid PK in the referenced table • Can be NULL (optional relationship) • Enables JOINS between tables • Prevents orphaned records

Example: *Orders.CustomerID → Customers.CustomerID*

Composite Key

A Primary Key made up of two or more columns. Used when no single column is unique enough.

Rules: Combination of columns must be unique • Common in junction/bridge tables • More complex to reference as FK • Avoid when a surrogate key works better

Example: *Enrollment(StudentID + CourseID), OrderItem(OrderID + ProductID)*



Natural Key: Uses existing business data (Email, SSN) | Surrogate Key: System-generated (AUTO_INCREMENT, UUID) — preferred in practice

RELATIONSHIP TYPES

One-to-One (1:1)

Each record in Table A relates to exactly ONE record in Table B, and vice versa.

e.g. *Person ↔ Passport* | *Employee ↔ Company Car* | *Country ↔ Capital City*

Implementation: FK in either table (usually the weaker entity) with UNIQUE constraint

Use when: *Use when splitting a large table for performance or security (e.g., user + user_credentials)*

Many-to-One (N:1)

Multiple records in Table A relate to ONE record in Table B (reading from A to B).

e.g. *Employees → Department* | *Orders → Customer* | *Products → Category*

Implementation: FK in the 'Many' side table pointing to the 'One' side PK

Use when: *Most common type — whenever multiple items share a single parent or owner*

One-to-Many (1:N)

ONE record in Table A relates to MANY records in Table B (the flip of N:1).

e.g. *Customer → Orders* | *Department → Employees* | *Category → Products*

Implementation: FK in the 'Many' side table (same as N:1 — it's just perspective!)

Use when: *Modelling parent-child relationships; hierarchy structures*

Many-to-Many (N:M)

Multiple records in Table A relate to MULTIPLE records in Table B.

e.g. *Students ↔ Courses* | *Orders ↔ Products* | *Actors ↔ Movies*

Implementation: Requires a JUNCTION TABLE with FKs to both tables (e.g., Enrollment, OrderItem)

Use when: *Any time both entities can have multiple associations with the other*

CARDINALITY & CROW'S FOOT NOTATION

Cardinality defines how many instances of one entity relate to another. The most widely used notation in industry is Crow's Foot.

Symbol	Meaning	Business Example	In SQL
(double bar)	Exactly One	Employee must have exactly 1 department	NOT NULL + UNIQUE FK
O (bar + circle)	Zero or One	Employee may have 0 or 1 company car	NULL FK
> (crow + bar)	One and Only One	Order must have exactly 1 customer	NOT NULL FK
>O (crow + circle)	Zero or Many	Customer may have 0 or many orders	NULL FK
> (crow + bar+bar)	One or Many	Department has 1 or many employees	NOT NULL FK with CHECK

Alternative: Min-Max Notation

Written as (min, max) beside each entity. **(0,1)** = optional one **(1,1)** = mandatory one **(0,N)** = optional many **(1,N)** = mandatory many

DESIGNING DATA FOR APPLICATIONS

Case Study: E-Commerce Platform

Entities & Key Attributes

Customer

CustomerID (PK), Name, Email, Phone, Address

Product

ProductID (PK), Name, Price, StockQty, CategoryID (FK)

Category

CategoryID (PK), CategoryName, Description

Order

OrderID (PK), CustomerID (FK), OrderDate, Status, Total

OrderItem

OrderItemID (PK), OrderID (FK), ProductID (FK), Qty, UnitPrice

Payment

PaymentID (PK), OrderID (FK), Method, Amount, PaidAt

Relationships

Customer → Orders

One customer places many orders

1:N

Order → OrderItems

One order has many line items

1:N

Product ↔ Orders

Via OrderItem junction table

N:M

Product → Category

Many products in one category

N:1

Order → Payment

One order has one payment record

1:1



Design tip: Always start with entities and relationships on paper/whiteboard BEFORE opening Draw.io or writing SQL.

ER DESIGN BEST PRACTICES



01 Avoid Redundancy (Normalize)

Don't store the same data in multiple places. If Customer Address appears in Orders, Customer, and Shipments — that's a design flaw. Store it once in Customers and reference via FK.



02 Use Surrogate Keys

Prefer system-generated integer IDs (AUTO_INCREMENT / UUID) over natural keys like Email or SSN. Natural keys change; surrogate keys don't.



03 Name Entities as Singular Nouns

Use 'Customer' not 'Customers'. Use 'Order' not 'Orders'. Consistent naming prevents confusion when writing SQL queries and joins.



04 Resolve Many-to-Many with Junction Tables

Never create M:N relationships directly. Always introduce a bridge/junction table with FKs to both entities. Add extra attributes to the junction table as needed.



05 Document All Relationships & Cardinality

Every relationship line must have notation showing min and max cardinality. A diagram without cardinality is incomplete — it leaves implementation ambiguous.



06 Validate Against Business Rules

Walk through real business scenarios: 'Can a product exist without a category?' 'Can an order have zero items?' The answers drive your cardinality choices and NOT NULL constraints.

COMMON ER DESIGN MISTAKES

✗ Missing Primary Keys

Every entity MUST have a PK. Without it, you cannot uniquely identify rows or create FK references.

✓ **Fix: Add a surrogate INT PK (id) to every table.**

✗ M:N Without Junction Table

Directly linking two tables in a Many-to-Many relationship is impossible in relational databases.

✓ **Fix: Create OrderItems, StudentCourses, ActorMovies bridge tables.**

✗ Ignoring NULL Semantics

'Optional' vs 'required' attributes must be defined up front — they map to NULL vs NOT NULL constraints.

✓ **Fix: For each attribute, explicitly decide: can this ever be empty?**

✗ Storing Derived Data

Don't store Age when you have DateOfBirth. Don't store OrderTotal when you can calculate $\text{SUM}(\text{UnitPrice} \times \text{Qty})$.

✓ **Fix: Store source data; derive values in queries.**

✗ Overloading Entities

Putting too many unrelated attributes into one entity (GodTable antipattern) kills performance and maintainability.

✓ **Fix: Split into related entities with proper FK relationships.**

✗ Skipping Business Validation

Designing only from technical needs, ignoring business rules like 'An order must always have a customer'.

✓ **Fix: Validate every cardinality decision against real scenarios.**

HANDS-ON LAB — DRAW.IO

Build the E-Commerce ER Diagram

Step 1

Open Draw.io

Go to draw.io → New Diagram → Entity Relationship template

Step 2

Add Entities

Create boxes for: Customer, Product, Category, Order, OrderItem, Payment

Step 3

Add Attributes

For each entity, add columns with data types (INT, VARCHAR, DECIMAL, DATETIME)

Step 4

Mark Keys

Underline Primary Keys. Mark Foreign Keys with (FK) notation

Step 5

Draw Relationships

Connect entities using Crow's Foot notation — set correct cardinality

Step 6

Validate

 Bonus Challenge: Add a Review entity (CustomerID FK, ProductID FK, Rating, Comment, ReviewDate) — what relationship type is this?

SESSION SUMMARY

Key Takeaways from S3: ER Diagrams

ER Diagrams: Are visual blueprints of databases — essential before writing a single line of SQL

3 Building Blocks: Entities (rectangles), Attributes (ovals), Relationships (diamonds)

Keys Matter: PKs uniquely identify rows; FKs enforce referential integrity between tables

4 Relationship Types: 1:1, N:1, 1:N, and N:M — each implemented differently in SQL

Crow's Foot Notation: Industry standard for expressing cardinality (minimum and maximum)

Design for Business Rules: Always validate every cardinality decision against real scenarios

Junction Tables: Required for ALL many-to-many relationships — never skip this step

Next Session → W13: ETL/ELT in Practice | Questions? Drop them in the chat!